

HOLY ANGEL UNIVERSITY
School of Computing
6ADET
Final Project Design System (Revised)

A Case Study Presented to the
Faculty of Mathematics and Science,
School of Arts and Sciences
Holy Angel University
Angeles City



Submitted by:
Allen David C. Panganiban

CS-302

August 2026

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

Bahantabay — Design System, Version 2

Community-Based Flood Monitoring and Route Warning App

Step A: The Palette, as a ColorScheme

Aa Flood Blue #1B4F72	Aa Warning Amber #E8A33D	Aa Flood Red #D64545	Aa Alice Blue #E6F1FB
Aa White #FFFFFF	Aa Mist Grey #F4F6FB	Aa Error Text #C23C3C	

I did not use `ColorScheme.fromSeed()` from a single seed. Bahantabay's colors carry specific meaning — Flood Blue is the brand identity, Warning Amber is caution, Flood Red is danger — and Material's auto-generated tonal palette would blend these into related shades rather than keeping them visually distinct. I hand-picked each role instead, written out below as the full scheme, with real hex values:

1. The role table, for reading:

Role	Hex	Used for
primary	#1B4F72	App bar, main buttons, active nav state
onPrimary	#FFFFFF	Text and icons on top of primary
secondary	#E8A33D	Warning badges, highlighted alerts, secondary actions
onSecondary	#1B2340	Text and icons on top of secondary (Ink Navy, not white — white on Amber fails contrast)
surface	#FFFFFF	Cards, sheets, dialogs
onSurface	#1B2340	Body text on cards and on the screen background

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

error	#D64545	"Not passable" status, destructive actions (large/bold text and icons only)
onError	#FFFFFF	Text and icons on top of error, restricted to large/bold use (see contrast check below)

Screen Backgrounds/Surface Roles

Role	Hex	Used for
App background	Mist Grey #F4F6FB	Default background for List View and normal functional screens
Primary / Flood Blue	#1B4F72	Primary actions, branding, Sign In hero background, and full-screen Home Map View
Surface	White #FFFFFF	Cards, sheets, dialogs, warning summary cards
Map Surface	Alice Blue #E6F1FB	Embedded map panels on Add Route, Route Details, and Report Flood

Home Map exception: Home uses two deliberately different presentation states. List View uses the normal Mist Grey application background, while Map View uses Flood Blue as an immersive full-screen map canvas. This does not introduce a new design-system color; it reuses the existing primary Flood Blue. Alice Blue remains the map-surface token for smaller embedded map panels inside Add Route, Route Details, and Report Flood.

Authentication screen treatment: Sign In / Guest Entry is the only screen that intentionally replaces the default Mist Grey application background with a full-bleed Flood Blue hero treatment. Its translucent circular decorations and frosted white/blue form surface are derived from the existing Flood Blue and White palette using transparency, so they do not introduce additional palette colors. Mist Grey remains the default background for functional screens after authentication, except for Home Map View, which intentionally uses the existing Flood Blue primary color as an immersive full-screen map canvas.

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

2. The Flutter object this palette becomes:

```
final appColorScheme = ColorScheme(
  brightness: Brightness.light,
  primary: Color(0xFF1B4F72),
  onPrimary: Color(0xFFFFFFFF),
  secondary: Color(0xFFE8A33D),
  onSecondary: Color(0xFF1B2340),
  surface: Color(0xFFFFFFFF),
  onSurface: Color(0xFF1B2340),
  error: Color(0xFFD64545),
  onError: Color(0xFFFFFFFF),
);
```

This is the full `ColorScheme()` constructor, not `fromSeed()` — I kept the manually-defined scheme because Flood Blue, Warning Amber, and Flood Red each carry a specific, intentional meaning (brand, caution, danger) that an auto-generated tonal palette from a single seed would blend together rather than preserve. The nine parameters above (brightness, primary, onPrimary, secondary, onSecondary, surface, onSurface, error, onError) are the fields `ColorScheme()` actually requires with no default in the current Material 3 API; other roles such as tertiary or primaryContainer fall back to sensible Material-generated defaults when omitted, and none of the five screens use them, so they are left out rather than invented.

3. Bahantabay-specific semantic colors (not standard `ColorScheme` roles):

```
class AppColors {
  static const warning = Color(0xFFE8A33D);
  static const errorText = Color(0xFFC23C3C);
  static const scaffoldBackground = Color(0xFFFF4F6FB);
  static const mapSurface = Color(0xFFE6F1FB);
  static const mutedText = Color(0xFF555555);
}
```

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

warning (#E8A33D) is the same hex as ColorScheme.secondary, kept as its own named constant because flood-status widgets (StatusBadge, FloodReportEntry) read AppColors.warning directly — that name states its meaning in the flood-monitoring domain, where ColorScheme.secondary states only its generic Material role.

errorText (#C23C3C) is a separate, darker red with no ColorScheme slot of its own, needed because Material only provides one error color and that one fails contrast at small text sizes (below). scaffoldBackground (#F4F6FB, Mist Grey) is set via ThemeData.scaffoldBackgroundColor, since Material 3 removed background/onBackground from ColorScheme in favor of surface.

Contrast check:

This is Bahantabay body text shown on the application background. Body text #1B2340 on #F4F6FB Contrast: 14.3:1 (Pass)	This is Bahantabay body text shown on a card surface. Body text #1B2340 on #FFFFFF Contrast: 15.4:1 (Pass)
Error: Please select a report location. Error text #C23C3C on #F4F6FB Contrast: 4.85:1 (Pass)	NOT PASSABLE White text on Flood Red #FFFFFF on #D64545 Contrast: 4.38:1 (Large/bold text only)

Ink Navy (#1B2340, onSurface) on Mist Grey background: 14.3:1. Ink Navy on White surface: 15.4:1. Both pass the 4.5:1 body-text minimum comfortably. One role fails: white text on Flood Red (error) measures 4.38:1, just under 4.5:1, so it is used only for large/bold text and icons, never small error copy. For small error text on white, I use the darker Error Text (#C23C3C, 5.2:1) instead, which passes.

Dark mode: light only.

Given the term timeline, I am not building a second color scheme. All colors are read from AppColors/ColorScheme, never hardcoded inline in a widget, so this file is the only place a color is

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

ever written literally — which is also what would make adding dark mode later a contained change instead of a search-and-replace across every screen.

Step B: The Type Scale, as a TextTheme

Code Snippet:

```
textTheme: TextTheme(

    headlineSmall: TextStyle(fontSize: 22, fontWeight: FontWeight.bold, color:
appColorScheme.onSurface),

    bodyMedium: TextStyle(fontSize: 16, color: appColorScheme.onSurface),

    labelSmall: const TextStyle(fontSize: 12, color: AppColors.mutedText,

),
```

Route Detail

Heading — headlineSmall, 22sp Bold — screen titles

St. Ignatius Subd. to Holy Angel University

Body — bodyMedium, 16sp Regular — normal text

Reported 10 min ago

Caption — labelSmall, 12sp Regular/Light — timestamps, hints

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

Your style	Flutter slot	Size	Weight	Used for
Heading	headlineSmall	22sp	Bold	Screen titles
Body	bodyMedium	16sp	Regular	Normal text
Caption	labelSmall	12sp	Regular/Light	Timestamps, hints

Used by name, never by literal size:

Text('Bahantabay', style: [Theme.of\(context\).textTheme.headlineSmall](#))

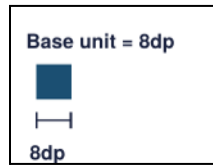
Only three styles, matching the original scale from the prelim — five simple screens do not need a fourth. Font family: Roboto (Flutter/Material default). No custom font is used. This keeps the typography consistent with the Material 3 system and avoids introducing an unnecessary dependency for the five-screen MVP. One distinct typeface was already enough to establish hierarchy across five screens; adding a custom font would be a cosmetic change with no functional payoff for this project. “Regular/Light” for Caption means regular font weight paired with the lighter Muted Text color, not a separate `FontWeight` value, matching `labelSmall`: `TextStyle(fontSize: 12, color: AppColors.mutedText)` with no font-weight override.

Step C: Spacing, as Constants

Base unit: 8dp **Screen edge padding: 24dp** **Gap between list items: 8dp** **Gap between sections: 16dp**

- 8dp — gap between list items (sm)
- 16dp — gap between sections, e.g. between route cards (md)
- 24dp — screen edge padding (lg)

HOLY ANGEL UNIVERSITY
School of Computing
6ADET



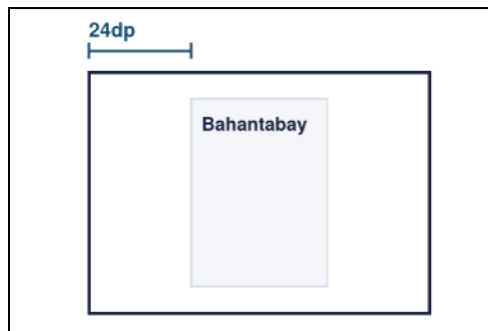
8dp - Base Unit



8dp - Gap between list items



16dp - Gap between sections



24dp - Screen edge padding

```
class AppSpacing {
    static const double xs = 4;
    static const double sm = 8;
    static const double md = 16;
    static const double lg = 24;
}
```

Used as: `padding: const EdgeInsets.all(AppSpacing.md)` — never a raw number typed inside a widget.

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

Step D: Components, as Files

The components below are the reusable or distinct buildable UI pieces identified from the five Bahantabay screens. Components that repeat across screens list all of their appearances; a screen-specific component is included when it represents a distinct widget rather than ordinary one-off text. Nothing is included simply to pad the component list.

Mockup reference text captured from the wireframes: "St. Ignatius Subd. to Holy Angel University", "2.4 km • Est. 12 min", "SAFE", "WARNING", "NOT PASSABLE", "St. Ignatius Road", "Knee-deep", "Not passable", "10 min ago", "Submit Report", "No saved routes yet".

Component	File	Constructor parameters	Appears on
RouteCard	lib/widgets/route_card.dart	String routeName, String startLabel, String endLabel, RouteStatus status, VoidCallback onTap	Home (route list), Route Details (summary header at the top of the screen)
StatusBadge	lib/widgets/status_badge.dart	RouteStatus status	Home, Route Details
FloodReportEntry	lib/widgets/flood_report_entry.dart	String location, String floodDepth, RoadStatus roadStatus, DateTime createdAt, VoidCallback? onTap	Home, Route Details
PrimaryButton	lib/widgets/primary_button.dart	String label, VoidCallback? onPressed, bool isLoading	Sign In/Guest, Add Route, Report Flood, Route Details
EmptyState	lib/widgets/empty_state.dart	String message, IconData icon	Home (no saved routes / no nearby reports), Route

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

			Details (no reports affecting this route)
RouteWarningBanner	lib/widgets/route_warning_banner.dart	String message	Route Details

Standard Material controls: Home's profile/account access uses a standard IconButton / PopupMenuButton in the app bar, while the List ↔ Map selector uses a Material SegmentedButton positioned directly below the app bar. These are standard Material controls rather than custom reusable components, so they are not added as separate component files.

RouteCard

lib/widgets/route_card.dart

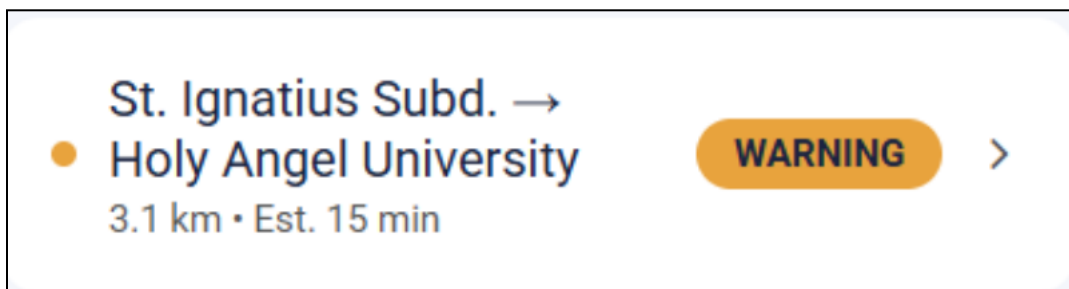
String routeName

String startLabel

String endLabel

RouteStatus status

VoidCallback onTap



Rendered from routeName and status; startLabel and endLabel compose the route line, while distance/time is derived data shown alongside. Route status is communicated through two

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

coordinated cues derived from the same RouteStatus: the text-labelled StatusBadge and a small leading status-color dot before the route name. The dot is a secondary visual cue only and never replaces the text-labelled badge.

StatusBadge

lib/widgets/status_badge.dart

RouteStatus status



Three states of the same widget, switched internally using RouteStatus.clear, RouteStatus.warning, and RouteStatus.notPassable. These are displayed to the user as SAFE, WARNING, and NOT PASSABLE. A text label always accompanies the status color so state never depends on color alone.

FloodReportEntry

lib/widgets/flood_report_entry.dart

String location

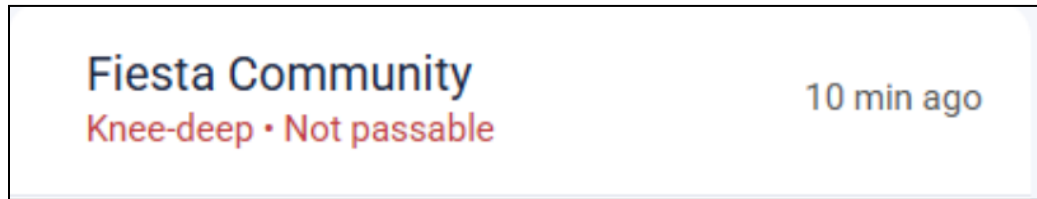
String floodDepth

RoadStatus roadStatus

DateTime createdAt

VoidCallback? onTap

HOLY ANGEL UNIVERSITY
School of Computing
6ADET



createdAt is formatted to a relative string (“10 min ago”) at render time; onTap is nullable, so the row still displays with no tap affordance when omitted.

PrimaryButton

lib/widgets/primary_button.dart

String label

VoidCallback? onPressed

bool isLoading



Same widget, two states: label text by default, a small spinner in place of the label when isLoading is true. onPressed is nullable so the button can render disabled.

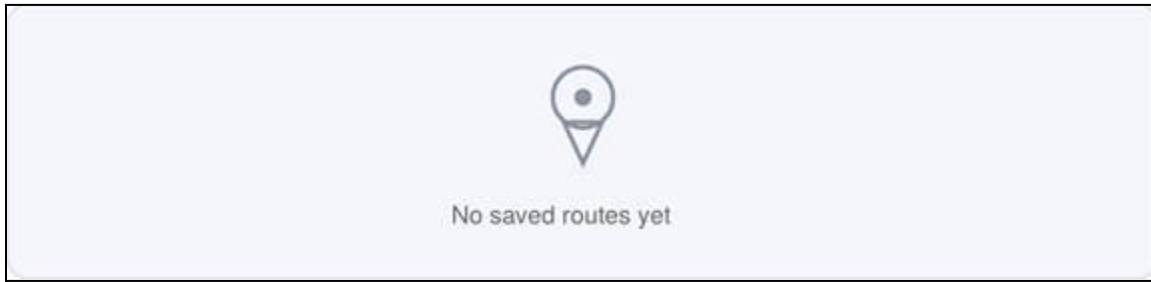
EmptyState

lib/widgets/empty_state.dart

String message

IconData icon

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

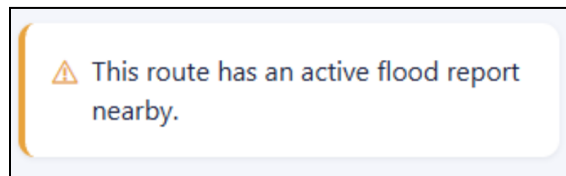


Centered icon + message only, no buttons or extra actions — matches the constructor, which takes no callback.

RouteWarningBanner

lib/widgets/route_warning_banner.dart

String message



Displays a prominent route-level warning using the supplied message. The amber treatment communicates caution while the written message ensures the warning does not depend on color alone.

Each component receives the data it needs through constructor parameters and, where interaction is required, exposes callbacks to the parent rather than managing screen state internally. Screen-level state is owned by the parent StatefulWidget and passed down to these components. Constructors use const wherever the parent allows it.

Step E: The Theme File, Assembled (bonus)

The following is the complete lib/theme.dart file that translates the design system into a reusable Flutter theme. It centralizes the application's colors, typography, spacing, card styling, and

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

button styling so the same visual rules can be applied consistently across the screens and reusable components.

```
// lib/theme.dart
```

```
import 'package:flutter/material.dart';
```

```
class AppColors {
```

```
  static const warning = Color(0xFFE8A33D);
```

```
  static const errorText = Color(0xFFC23C3C);
```

```
  static const scaffoldBackground = Color(0xFFFF4F6FB);
```

```
  static const mapSurface = Color(0xFFE6F1FB);
```

```
  static const mutedText = Color(0xFF555555);
```

```
}
```

```
class AppSpacing {
```

```
  static const double xs = 4;
```

```
  static const double sm = 8;
```

```
  static const double md = 16;
```

```
  static const double lg = 24;
```

```
}
```

```
final appColorScheme = ColorScheme(
```

```
  brightness: Brightness.light,
```

```
  primary: Color(0xFF1B4F72),
```

HOLY ANGEL UNIVERSITY**School of Computing
6ADET**

```

onPrimary: Color(0xFFFFFFFF),
secondary: Color(0xFFE8A33D),
onSecondary: Color(0xFF1B2340),
surface: Color(0xFFFFFFFF),
onSurface: Color(0xFF1B2340),
error: Color(0xFFD64545),
onError: Color(0xFFFFFFFF),
);

final appTheme = ThemeData(
  useMaterial3: true,
  scaffoldBackgroundColor: AppColors.scaffoldBackground,
  colorScheme: appColorScheme,
  textTheme: TextTheme(
    headlineSmall: TextStyle(
      fontSize: 22,
      fontWeight: FontWeight.bold,
      color: appColorScheme.onSurface,
    ),
    bodyMedium: TextStyle(
      fontSize: 16,
      color: appColorScheme.onSurface,
    ),
  ),
);

```

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

```
labelSmall: const TextStyle(
  fontSize: 12,
  color: AppColors.mutedText,
),
),
cardTheme: const CardThemeData(
  margin: EdgeInsets.all(AppSpacing.sm),
),
filledButtonTheme: FilledButtonThemeData(
  style: FilledButton.styleFrom(
    minimumSize: const Size.fromHeight(48),
  ),
),
);
```

The full Home Map View does not require a separate background color token. It reuses the existing Flood Blue primary color through `Theme.of(context).colorScheme.primary`, while `AppColors.mapSurface` is reserved for the lighter embedded map panels used on Add Route, Route Details, and Report Flood.

Theme.of(context).colorScheme.primary

The following is the complete lib/theme.dart file that assembles Step A's ColorScheme, Step B's type scale, and Step C's spacing constants into one reusable Flutter theme, plus AppColors for Bahantabay-specific semantic colors that do not require their own standard Material ColorScheme

HOLY ANGEL UNIVERSITY

School of Computing

6ADET

role. It centralizes colors, typography, spacing, card styling, and button styling so the same visual rules apply consistently across every screen and component.

What Changed, and Why

Element	Prelim said	Now says	Why it changed
Palette	AppColors constants, described as feeding into ColorScheme.fromSeed()	Same core palette, now expressed as a full hand-picked ColorScheme, with additional named semantic tokens for Map Surface (#E6F1FB) and Muted Text (#555555). Flood Blue is also explicitly documented as the Sign In / Guest Entry hero background while Mist Grey remains the default functional-screen background.	Building the M4A3 theme file showed me fromSeed() would blend Flood Blue, Warning Amber, and Flood Red into related tonal shades, losing the distinct meaning each color carries. Writing the full scheme by hand keeps that meaning intact, and separating semantic-only colors from Material roles keeps the ColorScheme itself standard-shaped.
Contrast	Flood Red flagged as borderline for small text, with a proposed darker variant	Confirmed: white-on-Flood-Red measures 4.38:1 and is restricted to large/bold use only; Error Text (#C23C3C, 5.2:1) is the fixed choice for small error text	I checked both combinations against a contrast tool this time instead of estimating, and kept the number rather than a vague "might fail" note.

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

Typography	Heading 22 / Body 16 / Caption 12, described generically	Same three sizes, mapped explicitly to <code>headlineSmall</code> / <code>bodyMedium</code> / <code>labelSmall</code> and used only via <code>Theme.of(context).textTheme</code> , never a literal <code>fontSize</code>	Shipping two graded apps in M4/M5 with inline <code>TextStyle(fontSize: ...)</code> made the size hard to find and change later; naming the slots fixes that going forward.
Spacing	8px base unit; 8/16/24 spacing rule, written as a guideline	Same rhythm, now an <code>AppSpacing</code> class with <code>xs/sm/md/lg</code> constants used inside <code>EdgeInsets.all(...)</code>	Same reason as typography: a rule on paper does not stop a hardcoded 16 from appearing in a widget; a named constant does.
Components	Described by appearance and props only, no file paths	Each component now has a concrete Dart file path and constructor signature. The mockup also revealed <code>RouteWarningBanner</code> as a distinct component on Route Details, so it is now formally documented rather than remaining one-off screen UI.	M4/M5 graded work required building reusable widgets this exact way (data in, callback out, no <code>setState</code> inside the widget), so these are written the way I now know they have to work.
RouteCard	Status was represented primarily by the <code>StatusBadge</code> .	<code>RouteCard</code> uses the <code>StatusBadge</code> plus a small leading status-color dot derived from the same <code>RouteStatus</code> .	The mockup showed that the second cue improves at-a-glance scanning. The badge remains the primary readable indicator, so accessibility does not depend on color.

HOLY ANGEL UNIVERSITY
School of Computing
6ADET

RouteWarni ngBanner	Not identified in the preliminary component system.	Added as RouteWarningBanner on Route Details.	Painting Route Details revealed that the relationship between the WARNING status and its nearby flood report needed an explicit explanatory element.
EmptyState	Not identified as a component in the prelim	Added as a component appearing on Home (no routes/no reports) and Route Details (no reports affecting the route)	Only became obvious once I actually laid out Home and Route Details for the mockup and noticed both need the same empty-list treatment; the prelim design system was written before those screens were fleshed out.
Auth Prompt Sheet	Listed as a component appearing on Home and Route Details when a guest attempts to report	Removed	It does not appear on any of the five current screens/wireframes; guest restrictions are handled by disabling actions, not an interrupting sheet. Keeping it would have listed a component no screen uses.
Home controls	Home navigation/account actions were not defined as part of the design system.	Home's app bar contains Bahantabay branding and a profile IconButton; a centered SegmentedButton below the app bar switches between List and Map.	Finalizing the mockup revealed both a missing sign-out/account-switch path and a clearer hierarchy for the view toggle. Standard Material

HOLY ANGEL UNIVERSITY**School of Computing****6ADET**

			controls solve both without adding a new screen or custom component.
Map treatment	Map surfaces were represented using one general map treatment.	Alice Blue remains the embedded-map surface, while Home Map View uses a full Flood Blue canvas.	Painting the complete Map View showed that Home benefits from a stronger geographic mode while Add Route, Route Details, and Report Flood still need lighter embedded map panels. Both treatments reuse established colors.